An Analysis of Probabilistic Data Structures for Scalable, Communication-Constrained Distributed Systems

Luca Lombardo

October 24, 2025

Abstract

Scalability in distributed systems depends fundamentally on managing the tradeoff between consistency guarantees and system performance. We analyze probabilistic data structures - Bloom filters, Distributed Bloom Filters, Cuckoo filters, and HyperLogLog - as architectural solutions to two canonical problems in large-scale systems: set membership testing and cardinality estimation. Through mathematical analysis, we establish that these structures achieve information-theoretic lower bounds on space complexity while providing quantifiable probabilistic guarantees. Bloom filters require $\Theta(n \log(1/\varepsilon))$ bits to maintain a set of n elements with false positive rate ε, independent of universe size $|\mathcal{U}|$. HyperLogLog estimates cardinality with standard error $\sigma = 1.04/\sqrt{m}$ using $\Theta(m)$ registers, independent of true cardinality n. We position these structures within the PACELC consistency framework, demonstrating that they instantiate the PA/EL configuration: prioritizing Availability during partitions and Latency during normal operation, accepting probabilistic Consistency. Communication complexity analysis reveals order-of-magnitude reductions compared to exact solutions: from $\Theta(n \log|\mathcal{U}|)$ to $\Theta(n \log(1/\varepsilon))$ for set reconciliation, and from $\Theta(Nn \log|\mathcal{U}|)$ to $\Theta(Nm \log\log|\mathcal{U}|)$ for distributed cardinality aggregation. The algebraic properties of merge operations (commutativity, associativity, idempotence) enable efficient distributed aggregation without centralized coordination. We conclude that probabilistic approximation represents a principled design choice in the consistency-performance tradeoff space, essential for systems scaling to billions of operations while maintaining bounded latency.

Introduction

The design of scalable distributed systems confronts a fundamental tension: exactness in computation conflicts with efficiency in communication. As distributed infrastructures grow to encompass thousands of nodes processing millions of requests per second [1], the quadratic growth in potential communication paths ($O(N^2)$ for N nodes) transforms network overhead from an optimization concern into an architectural constraint. Traditional approaches that guarantee exact answers - maintaining complete replicas, performing exhaustive synchronization, or transmitting entire datasets - become infeasible at scale.

This report examines a class of algorithmic solutions that resolve this tension through principled approximation: probabilistic data structures. These algorithms trade perfect accuracy for substantial reductions in space complexity and communication overhead, while providing quantifiable error bounds. We focus on structures addressing two canonical problems in distributed systems: set membership testing (Bloom filters, Distributed Bloom Filters, Cuckoo filters) and cardinality estimation (HyperLogLog). Our analysis establishes that these are not ad-hoc optimizations but instantiations of a coherent design philosophy positioned within the PACELC consistency framework.

Central Thesis and Contributions

We advance three interconnected claims:

Information-theoretic optimality. The analyzed probabilistic structures achieve space complexity matching theoretical lower bounds up to constant factors. For set membership with false positive rate ε, Bloom filters require $\Theta(n \log(1/\varepsilon))$ bits, matching the information-theoretic minimum. For cardinality estimation with relative error σ, HyperLogLog requires $\Theta(1/\sigma^2)$ space, again optimal. This optimality establishes that further improvements require either relaxing error guarantees

or solving different problems.

Architectural positioning in consistency models. Probabilistic structures instantiate the PA/EL quadrant of the PACELC theorem: they prioritize Availability during network partitions and Latency during normal operation, accepting probabilistic rather than strong Consistency. This positioning is not incidental but foundational - the communication savings that enable scalability derive precisely from relaxing consistency requirements. We formalize this relationship through comparative analysis with strong consistency protocols (Paxos, Raft), demonstrating order-of-magnitude reductions in message complexity.

Composability and algebraic structure. The efficiency of these structures in distributed settings stems from algebraic properties of their merge operations. HyperLogLog sketches combine via componentwise maximum - a commutative, associative, and idempotent operation. This algebraic structure enables tree-based aggregation, eliminating centralized coordination bottlenecks. We establish that such properties are essential for scalable distributed aggregation, not incidental implementation details.

Scope and Methodology

This report adopts a formal, mathematical approach. We present complete algorithmic descriptions with rigorous complexity analysis, including full proofs of key theorems. Each structure is analyzed along three dimensions: (1) space complexity and error bounds, (2) distributed communication patterns and message complexity, and (3) position within the PACELC consistency framework. We derive results from first principles rather than empirical observation, though we reference implementation studies where they provide architectural insights [2, 3].

The analysis assumes familiarity with probability theory, algorithm analysis, and distributed systems fundamentals. We employ standard complexity notation (O, Ω, Θ) and probabilistic analysis techniques. Proofs are provided for non-trivial results; straightforward derivations are stated without proof.

Organization

The report proceeds as follows. Section 1 establishes theoretical foundations: formal definitions of distributed systems and scalability, the PACELC consistency framework, precise problem statements for set membership and cardinality estimation, and information-theoretic lower bounds on space complexity. This groundwork enables subsequent sections to position specific algorithms within a unified theoretical framework.

Section 2 analyzes the Bloom filter family for set membership. We begin with the classical Bloom filter, deriving its false positive probability and optimal parameter selection. The Distributed Bloom Filter extends this to multi-node reconciliation through unique per-peer hash mappings. The Cuckoo filter addresses a fundamental limitation - support for deletions - through fingerprint storage and cuckoo hashing. We conclude with comparative analysis across space, time, and message complexity dimensions.

Section **??** examines cardinality estimation via HyperLogLog. We derive the algorithm from probabilistic counting principles, prove its accuracy bounds ($\sigma \approx 1.04/\sqrt{m}$), and analyze its distributed aggregation properties. The commutative merge operation enables efficient tree-structured computation, reducing communication from $\Theta(Nn\log|\mathcal{U}|)$ (exact counting) to $\Theta(Nm\log\log|\mathcal{U}|)$ (HyperLogLog), where typically $m \ll n$.

Section **??** provides synthesis through three lenses. First, we construct a comprehensive taxonomy classifying structures by mathematical properties (space-error tradeoffs), algorithmic properties (operation complexities), and distributed properties (merge semantics). Second, we position these structures within the PACELC framework, contrasting their PA/EL characteristics with PC/EC systems like Paxos. Third, we analyze real-world deployments, documenting how systems like Redis, PostgreSQL, and Apache Spark integrate probabilistic structures into production architectures.

The conclusion identifies open research directions: adversarial robustness, multi-dimensional tradeoff spaces, and hardware acceleration opportunities. We argue that as distributed systems continue scaling to billions of operations with millisecond latency requirements, probabilistic structures will transition from specialized techniques to architectural necessities.

1 Background and Theoretical Foundations

We define a distributed system as a collection of autonomous computing elements, or nodes, that appear to its users as a single coherent system. These nodes coordinate through message passing over a network to achieve a common objective, hiding the inherent distribution of resources and processing.

The challenge in designing distributed systems is scalability: the capacity to maintain performance and correctness as $|N|$, data volume, or request rate increases. Scalability is not merely an optimization objective but an architectural necessity that dictates system viability at scale.

1.1 The Communication Bottleneck

The primary impediment to scalability in large-scale distributed systems is communication overhead. Modern distributed infrastructures routinely manage thousands of nodes with millions of concurrent requests. When $|N| = k$ nodes must coordinate, the potential number of pairwise communications is $O(k^2)$, creating a bottleneck. The implications manifest in both bandwidth saturation and increased latency, directly constraining system throughput.

Theorem 1.1 (Quadratic Communication Complexity)**.** *In a fully-connected distributed system with k nodes where each node must exchange state with all others, the total message complexity is $\Theta(k^2)$ per coordination round.*

Proof. Each of the k nodes must send its state to $k-1$ other nodes. The total number of messages is $k(k-1) = \Theta(k^2)$. □

This quadratic growth establishes communication as the primary scaling bottleneck, motivating the design of communication-efficient algorithms.

1.2 The PACELC Consistency Framework

Distributed system design fundamentally navigates a space of architectural trade-offs. A foundational result, the CAP theorem, establishes an impossibility result: no distributed data store can simultaneously guarantee Consistency (all nodes observe identical state), Availability (all requests receive responses), and Partition tolerance (system functions despite network partitions). The PACELC theorem extends this framework to normal operation:

Definition 1.2 (PACELC Theorem)**.** A distributed system must make two independent choices: during network Partitions (P), choose between Availability (A) and Consistency (C); Else (E), during normal operation, choose between Latency (L) and Consistency (C). This defines four configuration quadrants: PA/EL, PA/EC, PC/EL, and PC/EC.

The PA/EL configuration prioritizes Availability during partitions and low Latency during normal operation, explicitly accepting relaxed consistency guarantees. The PC/EC configuration (exemplified by consensus protocols like Paxos and Raft) prioritizes Consistency in both scenarios at the cost of availability and latency.

This formulation reveals that consistency tradeoffs persist even in healthy systems. Every operation faces a choice: wait for strong consistency (higher latency) or return immediately with potentially stale data (lower latency). Probabilistic data structures make this choice explicit and quantifiable.

1.3 The Set Membership Problem

We now define the two canonical problems that motivate probabilistic data structures.

Definition 1.3 (Set Membership Problem)**.** Given a set $S \subseteq \mathcal{U}$ where $\mathcal{U}$ is a universe of elements, and a query element $x \in \mathcal{U}$, determine whether $x \in S$. The operations are INSERT(x) to add x to S, QUERY(x) to return whether $x \in S$, and optionally DELETE(x) to remove x from S.

Classical deterministic solutions, such as hash tables or search trees, require space proportional to the number and size of elements, which is prohibitive for large sets in communication-constrained systems.

Definition 1.4 (Set Reconciliation Problem)**.** Given two sets $S_i, S_j \subseteq \mathcal{U}$ residing on distinct nodes n_i, n_j, compute the symmetric difference:

$$\Delta(S_i, S_j) = (S_i \setminus S_j) \cup (S_j \setminus S_i) \tag{1}$$

without transmitting the complete sets.

This problem arises in applications such as distributed caching architectures [4], where nodes must efficiently determine coherence without exhaustive data comparison. Exact solutions require communication proportional to the set sizes, which is infeasible at scale.

1.4 The Cardinality Estimation Problem

Definition 1.5 (Cardinality Estimation Problem)**.** Given a multiset M (potentially distributed across k nodes as $M = M_1 \cup M_2 \cup \cdots \cup M_k$), estimate:

$$n = |\text{distinct}(M)| = |\{x : x \in M\}| \tag{2}$$

the number of distinct elements, without materializing the entire set at a single location.

This count-distinct problem is ubiquitous in distributed analytics: computing unique visitor counts in web analytics, identifying distinct user sessions across distributed logs, or determining the cardinality of a join operation in distributed query processing.

Exact computation across k nodes necessitates communication to aggregate all elements, which violates latency requirements at scale.

1.5 Space-Error Tradeoffs

We establish the relationship between space complexity and error for approximate solutions, as derived from the analyses in the source literature.

For the set membership problem, the analysis of the Bloom filter [5] shows that to achieve a target false positive rate ε for a set of n elements, the required space in bits is:

$$m = -\frac{n \ln \varepsilon}{(\ln 2)^2} \approx 1.44 n \log_2(1/\varepsilon) \tag{3}$$

This demonstrates a space complexity of $O(n \log(1/\varepsilon))$, a result that is known to be asymptotically optimal within a constant factor for this problem class.

For the cardinality estimation problem, the analysis of the HyperLogLog algorithm [6] shows that to achieve a standard error of σ, the required space m (number of registers) is:

$$\sigma \approx \frac{1.04}{\sqrt{m}} \implies m \approx \frac{1.08}{\sigma^2} \tag{4}$$

This implies a space requirement of $\Omega(1/\sigma^2)$ to achieve a desired relative error, a bound that is also known to be asymptotically optimal.

These results quantify the direct trade-off between the memory footprint of a probabilistic data structure and the precision of its answers.

1.6 The Probabilistic Approach

Both problems admit exact solutions but at prohibitive communication cost. We advance the thesis that probabilistic data structures provide a mathematically rigorous framework for implementing the PA/EL side of the PACELC trade-off. These algorithms accept a bounded error probability in exchange for substantial reductions in both space complexity and communication overhead.

The Bloom filter family addresses set membership with space complexity that is asymptotically optimal, as noted. The HyperLogLog algorithm estimates cardinality with a space-error tradeoff that is established as near-optimal [6].

Critically, these structures support efficient distributed operations. Bloom filters, for instance, enable probabilistic set reconciliation with communication proportional to the filter size, m, which is significantly smaller than the size of the original set. HyperLogLog sketches merge via a commutative, associative, and idempotent operation, enabling tree-structured aggregation without centralized coordination [7].

This analysis demonstrates that probabilistic approximation is not a concession to imperfection but a principled architectural strategy. When strong consistency is unnecessary for application semantics, these algorithms enable scalability that would otherwise be infeasible.

2 Probabilistic Structures for Set Membership

We consider the dictionary problem, which consists of maintaining a dynamic set $S \subseteq \mathcal{U}$ and supporting the operations INSERT(x), DELETE(x), and QUERY(x), where the QUERY(x) operation must return whether $x \in S$. In a large-scale distributed environment, this problem's complexity is amplified by the constraints of network communication. Classical deterministic solutions, such as hash tables or search trees, become infeasible. Such structures require space proportional to the number and size of elements, which is prohibitive

for massive sets. More critically, when a set is partitioned or replicated across multiple nodes to achieve scalability and fault tolerance, synchronizing these deterministic structures necessitates costly data transfers, creating communication overhead that represents the primary bottleneck to system performance [**?**].

Probabilistic data structures offer a solution by relaxing the correctness guarantee to achieve sublinear space complexity, a critical attribute for scalable systems. This approach allows a system to trade a quantifiable measure of precision for substantial gains in space and communication efficiency.

Definition 2.1 (One-Sided Error Membership Oracle)**.** A data structure $\mathscr{D}$ representing a set S is a one-sided error membership oracle with a false positive rate ε if for any element $x \in S$, the probability that $\mathscr{D}.\textsc{Query}(x)$ returns true is 1, and for any element $y \notin S$, the probability that $\mathscr{D}.\textsc{Query}(y)$ returns true is at most ε.

By definition, the oracle never produces false negatives. This property is paramount in distributed systems, as a definitive negative answer can be used to avoid expensive and high-latency remote operations, such as disk I/O or network round-trips [**?**]. The adoption of such structures is not an ad-hoc optimization but a principled architectural strategy. It provides a mechanism to implement the PA/EL configuration of the PACELC theorem, where a system, during normal operation, explicitly chooses lower Latency over strong Consistency. By accepting a bounded and well-defined error probability, these algorithms enable a level of scalability and performance that would be unattainable with exact, deterministic methods.

2.1 The Bloom Filter

The Bloom filter is a space-efficient data structure that provides an implementation of a one-sided error membership oracle [5].

Definition 2.2 (Bloom Filter)**.** A Bloom filter for a set S with cardinality $|S| \leq n$ from a universe $\mathscr{U}$ consists of a bit array B of length m initialized to all zeros, and a collection of k independent hash functions $\mathscr{H} = \{h_1, \ldots, h_k\}$, where each $h_i : \mathscr{U} \to \{0, 1, \ldots, m-1\}$.

The insertion and query operations are defined as follows. To insert an element $x \in S$, the bits at positions $B[h_i(x)]$ are set to 1 for all $i \in \{1, \ldots, k\}$. To query for an element y, the operation returns true if and only if $B[h_i(y)] = 1$ for all $i \in \{1, \ldots, k\}$. A query for any element $y \in S$ is guaranteed to return true, as all corresponding bits will have been set during insertion, thus satisfying the no-false-negatives condition of the oracle.

For an element $y \notin S$, a false positive occurs if all k bits corresponding to its hash values have been incidentally set to 1 by the insertion of other elements. Assuming the hash functions distribute elements uniformly over the bit array, the probability of a specific bit remaining 0 after n insertions is $(1 - 1/m)^{kn} \approx e^{-kn/m}$. The probability of a false positive, ε, is therefore approximated by the probability that all k bits for the queried element are 1 [5].

Proposition 2.3 (False Positive Probability)**.** *The false positive probability of a Bloom filter is* $\varepsilon \approx (1 - e^{-kn/m})^k$.

Given a fixed ratio of bits per element, m/n, there exists an optimal number of hash functions, k, that minimizes this probability.

Theorem 2.4 (Optimal Hash Function Count)**.** *For a given ratio m/n, the false positive rate ε is minimized when the number of hash functions is $k = (m/n) \ln 2$.*

This leads to a direct relationship between the space allocated per element and the achievable false positive rate, which defines the fundamental space-error tradeoff of the structure.

Corollary 2.5 (Space-Error Relationship)**.** *To achieve a target false positive rate ε with an optimal number of hash functions, the required number of bits per element is given by* $m/n = -\frac{\ln \varepsilon}{(\ln 2)^2} \approx 1.44 \log_2(1/\varepsilon)$.

This result establishes the asymptotic space complexity of the Bloom filter as $\Theta(n \log(1/\varepsilon))$, independent of the size of the elements or the universe $\mathscr{U}$. The performance and accuracy of the filter depend on the quality of its hash functions. While universal hashing provides theoretical guarantees, fast non-cryptographic hash functions with good distribution properties, such as Murmur3, are often used in practice. Techniques like double hashing, which generates k hash values from two initial hashes, can further reduce computational overhead without increasing the asymptotic false positive probability [5].

Despite its efficiency, the standard Bloom filter has two fundamental limitations that restrict its use in dynamic distributed systems. First, it does not support element deletion; clearing a bit from 1 to 0 could inadvertently

affect other elements that map to that bit, thereby introducing false negatives. Second, its size is fixed at creation time based on an expected number of elements, n. If the actual number of elements substantially exceeds this estimate, the false positive rate degrades rapidly [**?**, 5].

2.1.1 Bloom Filter Variants for Dynamic Systems

The static nature of the standard Bloom filter creates significant operational challenges in dynamic distributed systems where set cardinalities are unknown or fluctuate. We analyze variants that address these limitations.

2.1.2 Counting Bloom Filter

To support element deletion without introducing false negatives, the Counting Bloom Filter (CBF) modifies the underlying structure [5].

Definition 2.6 (Counting Bloom Filter)**.** A Counting Bloom Filter consists of an array C of m counters, each of c bits, initialized to all zeros. It uses a collection of k independent hash functions $\mathcal{H} = \{h_1, \ldots, h_k\}$, where each $h_i : \mathcal{U} \to \{0, \ldots, m-1\}$.

The operations are redefined to act upon these counters. To insert an element, the counters at the k hash positions are incremented. To delete an element, the corresponding counters are decremented. A membership query returns true if and only if all corresponding counters are positive. The primary trade-off is a space complexity increase by a factor of c compared to a standard Bloom filter. The selection of counter size c must be sufficient to avoid overflow; for most applications, a 4-bit counter is sufficient [5]. The utility of the CBF in distributed systems extends to set reconciliation protocols, where the arithmetic properties of the counters allow for the computation of set differences by subtracting one filter's counter array from another [**?**].

2.1.3 Scalable Bloom Filter

To accommodate sets of unknown or unbounded cardinality, the Scalable Bloom Filter (SBF) employs a sequence of standard filters [5].

Definition 2.7 (Scalable Bloom Filter)**.** An SBF is a sequence of s Bloom filters, $B_1, B_2, \ldots, B_s$. Each filter B_i is defined by its size m_i and number of hash functions k_i. The sequence is constructed such that for $i > 1$, $m_i > m_{i-1}$ and the target false positive rate $\varepsilon_i < \varepsilon_{i-1}$, typically following a geometric progression.

New elements are inserted only into the last filter in the sequence, B_s. If this filter exceeds a predefined capacity threshold, a new filter B_{s+1} with stricter parameters is appended. A query operation must check for the element's presence in every filter from B_s down to B_1. While this architecture allows the structure to grow dynamically, its application in distributed systems is limited by its lack of simple composability. The heterogeneity of the constituent filters prevents a meaningful bitwise union operation, complicating state aggregation tasks that rely on the algebraic properties of standard Bloom filters [**?**, 5].

2.1.4 Variants for Parallelism and Network Efficiency

Further variants have been developed to address the operational demands of high-concurrency and communication-constrained environments.

2.1.5 Parallelism and Composability

A Partitioned Bloom Filter modifies the internal structure by dividing the bit array into k disjoint slices, with each hash function mapped to a unique slice. This can facilitate parallel operations by reducing contention [**?**, 5]. A more advanced architecture, the Parallel Partitioned Bloom Filter (Par-BF), is designed for high-throughput dynamic systems. It uses a series of homogeneous sub-filters (sharing the same m and k). This homogeneity preserves the ability to perform bitwise union and intersection operations, a feature lost in the SBF. The list of sub-filters is then partitioned into groups, allowing query operations to be executed in parallel across these groups, thus regaining operational flexibility for tasks like state merging and garbage collection in distributed settings [**?**].

2.1.6 Network Efficiency

The Compressed Bloom Filter is designed to optimize the transmission of filter state over a network. The key idea is to use a non-optimal number of hash functions k such that the resulting bit array has a lower probability of bits being set to one (less than $1/2$). This makes

the bit array more amenable to standard compression algorithms, potentially resulting in a smaller transmitted size for a given target false positive rate after decompression [5].

2.1.7 Challenges in Distributed Bloom Filter Implementations

The deployment of Bloom filters in a distributed system introduces a new class of complex challenges that are absent in single-node implementations. These challenges arise from the fundamental realities of distributed computing: network latency, the possibility of network partitions, and the need to manage a shared state across independent nodes.

2.1.8 Synchronization, Staleness, and the False Negative Paradox

In many distributed architectures, it is advantageous to replicate a Bloom filter across multiple nodes to allow for fast, local lookups. However, maintaining perfect, instantaneous consistency across these replicas is physically impossible due to network latency. This inherent delay means that replicas will inevitably become "stale" [**?**, **?**].

The most profound consequence of replica staleness is the violation of the Bloom filter's core guarantee: the absence of false negatives. In a distributed system with replicated filters, a new type of error emerges. Consider a system where an element x is added to the global set. The update is applied to the primary Bloom filter on Node A, and a message is sent to update the replica on Node B. Due to network propagation delay, there is a time window during which Node A's filter contains x but Node B's replica does not. If a client queries the stale replica on Node B for element x during this window, the replica will return false. However, from the perspective of the global system state, x is a member of the set. The system has therefore returned a definitive "no" for an element that actually exists. This is a temporal false negative, induced by the temporal inconsistency of the distributed state [**?**]. This phenomenon has severe implications, as many systems are architected around the assumption that a false from a Bloom filter is an authoritative answer.

2.1.9 Managing the Global False Positive Rate

In a single-instance Bloom filter, the false positive rate (FPR) is a predictable probability. In a distributed system composed of multiple, potentially desynchronized filters, the "global" or "effective" FPR becomes a complex, emergent property. Several factors contribute to this complexity. Out-of-sync replicas degrade the system's overall accuracy. In protocols where filters are merged (e.g., via a bitwise OR), the FPR of the resulting filter depends non-trivially on the degree of overlap in the element sets. In a partitioned system, data skew can cause filters on "hot" partitions to fill up much faster than others. Their individual FPRs will degrade more quickly, and these overloaded filters will disproportionately contribute to the overall system's effective FPR. This complexity means that managing the FPR in a distributed system must transition from a static, upfront calculation into a dynamic, operational monitoring problem [**?**].

2.1.10 Architectural Patterns and Protocols for Synchronization

To manage the complexities of distribution, specific architectural patterns have been developed.

2.1.11 Models for Filter Distribution

Three primary models are prevalent, each embodying a different set of trade-offs between consistency, availability, and performance. A *Centralized Model* uses a single, logically centralized Bloom filter (e.g., implemented in a service like Redis) as the source of truth. This simplifies consistency but introduces a potential bottleneck and single point of failure [**?**]. A *Partitioned Model* uses techniques like consistent hashing to map each element to a specific node, which maintains a filter for its local data partition. This architecture is highly scalable and partition-tolerant [**?**, **?**]. A *Replicated Model with Event-Driven Updates* maintains a local replica of the global filter on each node, providing the lowest possible read latency. Updates are propagated asynchronously via a messaging system (e.g., Kafka). This model prioritizes read availability but is most susceptible to the temporal false negative paradox due to propagation delays [**?**, **?**]. The selection among these models is a direct application of the trade-offs described by the CAP and PACELC theorems, align-

ing the filter architecture with the fundamental priorities of the application.

2.1.12 Protocols for Efficient Set Reconciliation

In eventually consistent systems, nodes must periodically synchronize their datasets. Transmitting full datasets is prohibitively expensive. Set reconciliation protocols use compact data structures to identify and transmit only the differences. Bloom filters, and particularly Counting Bloom Filters, can be used to probabilistically identify missing or extra elements between two nodes [**?**, **?**]. For this specific problem, the Distributed Bloom Filter (DBF) protocol provides a highly communication-efficient solution for guaranteeing eventual consistency [**?**].

Definition 2.8 (Distributed Bloom Filter Protocol)**.** Let two nodes, n_i and n_j, with unique identifiers ID_i and ID_j, wish to reconcile their local sets S_i and S_j. The DBF protocol employs a unique, pair-wise hash function family $\mathscr{H}_{ij} = \{H_1^{ij}, \ldots, H_k^{ij}\}$. This family is generated using a seed s_{ij}, typically computed as $s_{ij} = ID_i \oplus ID_j$. A set of k intermediate hashes $\{h_1^{ij}, \ldots, h_k^{ij}\}$ is derived from s_{ij}. For each element $x \in \mathscr{U}$ with a precomputed hash $H(x)$, the final hash values are computed as:

$$H_l^{ij}(x) = (H(x) \oplus h_l^{ij}) \pmod{m} \quad \forall l \in \{1, \ldots, k\}$$

This computationally inexpensive method allows for the rapid generation of unique bloom filters for each peer interaction.

The protocol deliberately uses small filters with a high individual false positive rate (e.g., 50%) to minimize bandwidth. While a single exchange is inaccurate, the use of unique mappings for each interaction ensures that hash collisions are statistically independent across successive rounds. True set differences are revealed consistently, while random errors from false positives average out. This sacrifices single-shot accuracy for rapid long-term convergence [**?**].

Proposition 2.9 (DBF System-Wide Error Rate)**.** *Let a system of N nodes use the DBF protocol, where each individual filter has a false positive rate ε. For an element $y \notin \bigcup_{i=1}^{N} S_i$, the probability that a query for y from a specific node to all of its $N-1$ peers results in a system-wide false positive is $\varepsilon_{System} = \varepsilon^{N-1}$.*

Proof. Let $\varepsilon = (1 - e^{-kn/m})^k$ be the FPR for a single filter representing a set of size n. A system-wide false positive for a query from node n_j regarding an element y occurs if all $N-1$ peers it communicates with independently return a false positive. Let E_i be the event of a false positive from peer n_i. The DBF protocol's use of unique seeds s_{ji} for each peer interaction ensures that the hash function families $\mathscr{H}_{ji}$ are statistically independent for each peer i. Consequently, the events E_i are independent. The probability of a joint failure across all peers is the product of their individual probabilities. Therefore, $P(\bigcap_{i=1}^{N-1} E_i) = \prod_{i=1}^{N-1} P(E_i) = \varepsilon^{N-1}$. □

This result demonstrates that even with a high individual filter error rate ε, the probability that all peers independently produce a false positive for the same element becomes vanishingly small as the number of peers increases, making the DBF an effective protocol for scalable and bandwidth-efficient set reconciliation [**?**].

2.2 Cuckoo Filters

The transition from single-node to distributed deployment exposes a structural limitation in Bloom filters that cannot be overcome through parameterization or minor refinement. In distributed systems, dataset cardinality is neither fixed nor predictable. Nodes experience churn; data undergoes compaction and replication; entries acquire expiration semantics. Elements must be deleted, not merely left to decay. A Counting Bloom Filter 2.6 addresses deletion through per-position counters, but incurs approximately four-fold space overhead compared to standard Bloom filters [5]. More fundamentally, neither variant preserves the algebraic properties required for distributed state reconciliation: the ability to compute set differences and identify true absence versus deletion.

The Cuckoo Filter presents a different architectural path. It natively supports efficient deletion in $O(1)$ time without counters or additional metadata. Furtehrmore, for false positive rates below three percent, it consumes less space than optimized Bloom filters [8]. Critically, these properties enable design patterns specifically suited to distributed scalability such as: elastic capacity growth without service interruption, co-location of filters with data partitions, and consistency models that distinguish true absence from deleted membership.

2.2.1 Partial-Key Hashing

The fundamental challenge in deploying Cuckoo Filters across network boundaries lies in the relocation operation itself. During insertion, when both candidate buckets are full, the filter must displace an existing fingerprint to its alternate location. Standard cuckoo hashing accomplishes this by recomputing the hash of the original element; the element's full key must be available to determine where to move it. In a distributed system, however, filters may be partitioned or replicated across nodes, and the original element may not be co-located with the filter metadata. Retrieving the full element for relocation decisions across network boundaries would introduce prohibitive overhead.

The solution is partial-key cuckoo hashing, introduced by Fan et al., which enables relocation decisions using only the stored fingerprint [8]. This capability is the prerequisite for all subsequent distributed patterns.

Definition 2.10 (Partial-Key Cuckoo Hashing [8])**.** For an element $x \in \mathcal{U}$ with precomputed fingerprint $f = \text{FINGERPRINT}(x)$, the two candidate bucket indices are computed as:

$$i_1 = \text{HASH}(x), \quad i_2 = i_1 \oplus \text{HASH}(f)$$

where $\oplus$ denotes bitwise XOR. There is a symmetry that allows us to, given i_2 and f, recover i_1 via the same equation: $i_1 = i_2 \oplus \text{HASH}(f)$.

This symmetry decouples relocation from full-key access. When a fingerprint must be moved from bucket i_1 to its alternate location, the system computes the destination using only f and the current position. The fingerprint can traverse network boundaries, be temporarily replicated on different nodes, or be moved as part of partition rebalancing, all without requiring access to the original element. For distributed systems where filters may be stored separately from data, or where data undergoes continuous replication and migration, this property is essential.

Definition 2.11 (Cuckoo Filter Core Operations [8])**.** A Cuckoo Filter with m buckets, each containing b entries, supports three operations:

Lookup(x): Compute $i_1 = \text{HASH}(x)$, $i_2 = i_1 \oplus \text{HASH}(f)$. Return true if fingerprint f exists in either bucket i_1 or i_2; otherwise return false. Query always examines at most $2b$ entries.

Insert(x): Compute i_1, i_2. If either bucket contains an empty entry, place f there and complete. Otherwise, select one bucket randomly, displace its resident fingerprint f', compute the alternate location for f' using $i \leftarrow i \oplus \text{HASH}(f')$, and repeat the displacement process. If relocations exceed a configurable threshold (typically 500), insertion fails and a capacity increase is required.

Delete(x): Compute i_1, i_2. If f exists in either bucket, remove one copy; otherwise return failure. Deletion requires no counters or additional bookkeeping.

We analyze the probability of a false positive, which occurs when an element not in the set hashes to a fingerprint that collides with an existing fingerprint in one of its two candidate buckets. A query inspects at most $2b$ entries across two buckets. This inspection process yields a direct upper bound on the false positive probability, which we state formally.

Proposition 2.12 (False Positive Probability Bound [8])**.** *Let a Cuckoo Filter be configured with b entries per bucket and a fingerprint length of f bits. The false positive probability ε satisfies the inequality:*

$$\varepsilon \leq \frac{2b}{2^f}$$

From this inequality, we derive the minimum fingerprint size f required to achieve a target false positive probability ε:

$$f \geq \lceil \log_2(1/\varepsilon) + \log_2(2b) \rceil$$

For a configuration with $b = 4$ and a target $\varepsilon = 0.01$ (a 1% false positive rate), this relationship requires a fingerprint size of at least $f \geq \lceil \log_2(100) + \log_2(8) \rceil = \lceil 6.64 + 3 \rceil = 10$ bits. This yields a compact representation that is computationally sufficient to mitigate a high rate of collisions.

The architectural consequences of this design become evident when we consider the delete operation. A standard Bloom filter offers no delete mechanism; removing an element's bits risks corrupting the representation of other stored elements. A Counting Bloom Filter enables deletion but requires per-position counters, which results in a space overhead of approximately four times that of a standard Bloom filter [8]. In contrast, the Cuckoo Filter implements deletion by locating the specified fingerprint in one of its two candidate buckets and removing it. This operation requires no auxiliary data structures and directly enables the dynamic distributed architectures we analyze subsequently.

2.2.2 Capacity Scaling

In a distributed system, the cardinality of a dataset is not known *a priori*, requiring any data structure to accommodate significant growth. A Cuckoo Filter of a fixed size cannot accept new insertions once its load factor approaches its theoretical limit. While single-node deployments address this by rebuilding the filter into a larger table offline, such a procedure is untenable in distributed systems that operate under strict availability and latency constraints. A node that halts operations to rebuild its local filter violates service agreements and introduces a point of coordination that undermines distributed design principles.

The Dynamic Cuckoo Filter (DCF), introduced by Chen et al., presents a systematic solution to this scaling problem [9]. Instead of rebuilding a full filter, the DCF architecture allocates a new, empty filter and appends it to an ordered sequence of filters. All new insertions are directed to the most recently appended filter.

Definition 2.13 (Dynamic Cuckoo Filter [9])**.** A DCF is an ordered sequence of Cuckoo Filters, $\mathrm{CF}_1, \mathrm{CF}_2, \ldots, \mathrm{CF}_k$. A pointer, curCF, references the terminal filter in the sequence. An insertion operation first attempts placement on curCF. If this insertion fails due to capacity limits, a new filter is allocated, appended to the sequence, and designated as the new curCF. A query operation must traverse this sequence in reverse chronological order, from curCF to CF_1, until the target fingerprint is found or all filters have been examined.

The DCF architecture permits unbounded capacity growth without service interruption, but it achieves this at the cost of query performance. A query for an element x requires a sequential search through the filter chain. If x was inserted into an early filter and the chain has subsequently grown to a length of k, the query must inspect all k filters.

Proposition 2.14 (DCF Query Complexity)**.** *A membership query in a DCF composed of k constituent filters requires $O(k)$ bucket accesses in the worst case. For a system containing n total items, where each filter maintains a load factor $\alpha \approx 0.95$ and has a capacity of C items, the number of filters scales as $k = \Theta(n/C)$. Consequently, query latency grows proportionally with the total dataset size.*

This architectural pattern creates a direct conflict between capacity scaling and query performance. The mechanism that enables system growth is the same mechanism that causes query latency to degrade. For modern distributed systems where query latency is a critical performance metric—such as in data serving, caching, and real-time analytics—a linear growth in query time is architecturally unacceptable.

The linear query complexity arises because the DCF organizes its constituent filters temporally, by insertion order, rather than deterministically by element hash value. No direct mapping exists between an element's content and the specific filter that contains it. Membership testing therefore degenerates to a sequential search, an operation incompatible with the constant-time query requirements of scalable distributed systems.

2.2.3 Consistent Hashing

The linear degradation in query performance of the Dynamic Cuckoo Filter arises from its temporal organization. We observe that distributed systems have already addressed analogous scaling challenges for data partitioning. Architectures such as Apache Cassandra, Redis Cluster, and DynamoDB employ consistent hashing to distribute their keyspace [10, 11]. In these systems, nodes are assigned token ranges on a virtual hash ring. An element's hash value deterministically maps it to a specific point on the ring, and thus to a responsible node. The addition or removal of a node requires remapping only a localized subset of the keyspace, preserving a stable and deterministic mapping for the majority of data.

This same principle can be applied to the organization of a Cuckoo Filter's buckets. Instead of organizing filters in a temporal sequence, we can distribute the buckets themselves across a consistent hash ring. This architectural shift aligns the filter's structure with established distributed partitioning schemes.

Definition 2.15 (Index-Independent Cuckoo Filter [12])**.** An Index-Independent Cuckoo Filter (I2CF) maintains its buckets on a consistent hash ring. For an element x with fingerprint f, its candidate bucket locations are determined by hashing the element to points on this ring. The system identifies the first bucket successor to each of these points on the ring as a candidate bucket.

The defining property of the I2CF is that bucket identifiers depend only on the element's hash value, not

on the total number of buckets m in the system. Capacity expansion is achieved by adding a new bucket to a chosen position on the ring. This operation necessitates relocating only those elements whose hash values fall within the range immediately preceding the new bucket's position. For a system with n items distributed across m buckets, a rebalancing event affects an expected $O(n/m)$ items, a small fraction of the total dataset.

This design decouples capacity growth from query complexity, resolving the architectural conflict present in the DCF.

Theorem 2.16 (I2CF Query Complexity)**.** *For an I2CF with m buckets organized on a consistent hash ring, a membership query for an element x requires a sequence of constant-time operations: (1) hash computations to determine ring positions, (2) a ring lookup to identify candidate buckets (achievable in $O(1)$ expected time with an appropriate auxiliary index), and (3) inspection of entries within those buckets. The total query complexity is therefore $O(1)$, independent of the total number of items n and the total number of buckets m.*

The I2CF architecture resolves the conflation of capacity management and element mapping found in the DCF. The DCF modifies the element-to-filter mapping to accommodate growth, leading to performance degradation. In contrast, the I2CF maintains a fixed, deterministic mapping from elements to ring positions while allowing the set of buckets to expand elastically. As a result, growth and query performance become orthogonal concerns.

2.2.4 Co-Partitioning Filters with Distributed Data

We observe that the architectural value of the Index-Independent Cuckoo Filter is most fully realized when its partitioning scheme is aligned with that of the host distributed system. Many distributed key-value stores partition their keyspace across a set of nodes using a consistent hash ring. In such systems, each node assumes responsibility for one or more token ranges on the ring.

We apply this same partitioning logic to the filter's buckets, an approach whose efficacy is demonstrated by Luo et al. in their work on distributed filters over Redis Cluster [3]. We formalize this alignment as a co-partitioned filter architecture.

Definition 2.17 (Co-Partitioned Filter Architecture [3])**.** A distributed system of m nodes maintains a consistent hash ring, where each node n_i is assigned responsibility for a token range $[t_i, t_{i+1})$. The node maintains: (1) all data items d such that $H(d) \in [t_i, t_{i+1})$, and (2) all Cuckoo Filter buckets b such that position$(b) \in [t_i, t_{i+1})$. A membership query for an element x is routed to the node responsible for the token range containing $H(x)$ for local processing.

This architecture has direct performance implications. A query processed on the node that owns the element's token range incurs only the cost of local memory access. A query originating from a different node requires a single network round-trip. The latency difference between local memory access (microseconds) and network communication (milliseconds) is typically three to four orders of magnitude.

We can model the expected query latency for this architecture under a uniform query distribution. In a system with m nodes, a randomly routed query has a probability $1/m$ of being processed locally and a probability $(1-1/m)$ of requiring network transit. This leads to the following formulation for the expected query latency.

Theorem 2.18 (Co-Partitioned Query Latency)**.** *In a co-partitioned I2CF architecture of m nodes under a uniform query distribution, the expected query latency $\mathbb{E}[L_q]$ is given by:*

$$\mathbb{E}[L_q] = \frac{1}{m} t_{local} + \left(1 - \frac{1}{m}\right) t_{network}$$

where t_{local} represents local bucket access time and $t_{network}$ represents network round-trip latency.

This latency bound is independent of the total dataset size n. Query performance is determined by hardware characteristics and system scale (m), not by the number of elements stored.

The alignment creates a synergy with the underlying infrastructure of many distributed systems. By reusing an existing consistent hashing ring, the system gains a scalable membership testing capability with minimal additional coordination overhead. Fault tolerance mechanisms, such as data replication, can be extended to include filter buckets, allowing replicated filter state to propagate through existing replication channels.

The experimental results in [3] confirm the efficacy of this approach, demonstrating significant memory savings over Bloom filter alternatives, full support for dele-

tions, and seamless capacity scaling as nodes are added to the cluster.

2.2.5 Hierarchical Elasticity for Non-Uniform Growth Rates

While the I2CF architecture provides fine-grained, bucket-level elasticity, its rebalancing mechanism is most effective under predictable, incremental growth in dataset cardinality. Distributed systems, however, often experience non-uniform growth patterns, including sudden spikes in insertion rates. In such scenarios, where the rate of cardinality growth exceeds the rate at which new buckets can be added and populated, the system may experience cascading insertion failures.

To address this class of workloads, we analyze the Consistent Cuckoo Filter (CCF), an architecture that introduces a second, coarser tier of elasticity by organizing a collection of independent I2CF instances [12].

Definition 2.19 (Consistent Cuckoo Filter [12])**.** A CCF is a collection of I2CF instances, $\{I_1, I_2, \ldots, I_s\}$, initially with $s = 1$. Insertions are directed to a designated active instance, typically I_s. When I_s reaches a predefined load factor threshold, a new, empty instance I_{s+1} is allocated and becomes the active target for subsequent insertions. A query operation must inspect all instances in the collection until the target fingerprint is found.

This architecture allows the system to increase its total capacity instantly by adding a new, untapped I2CF. However, this "scale-out" mechanism, if left unmanaged, structurally resembles the chaining model of the Dynamic Cuckoo Filter. A query must probe every instance in the collection, leading to a performance dependency on the number of instances, s.

Proposition 2.20 (CCF Unmanaged Query Complexity)**.** *In a CCF where new, fixed-capacity I2CF instances are added sequentially without any consolidation, the number of instances s grows linearly with the total number of items n. The worst-case query complexity is therefore $O(s)$, which is equivalent to $O(n)$.*

Proof. Let n be the total number of items and let each I2CF instance have an effective capacity of C items (e.g., $C = m \cdot b \cdot \alpha$, where m is the number of buckets, b is the entries per bucket, and α is the target load factor). In an unmanaged growth model, a new instance is added every time the system has stored an additional C items. The number of instances s required to store n items is therefore $s = \lceil n/C \rceil$. As $n \to \infty$, this gives $s = \Theta(n)$. Since a query must inspect all s instances in the worst case, the query complexity is $O(s) = O(n)$. □

The CCF architecture as defined by Luo et al. explicitly includes management policies to prevent this linear degradation [12]. It is not merely a chain of filters but a managed system that provides hierarchical scaling. This system operates at two granularities. The first is a fine-grained, bucket-level elasticity, where capacity adjustments occur within each constituent I2CF by adding or removing buckets from its consistent hash ring. The second is a coarse-grained, filter-level elasticity, which accommodates sudden load spikes by adding entire new I2CF instances. To control the growth of query latency, the architecture incorporates a consolidation mechanism, which periodically merges underutilized instances to reduce the total number of instances, s. This hierarchical approach allows the system to manage capacity at multiple granularities, handling gradual growth through the fine-grained mechanism and absorbing sudden traffic spikes by adding new instances, while maintaining long-term performance through background consolidation.

2.2.6 Concurrency Control Strategies for Distributed Cuckoo Filters

The deployment of Cuckoo Filters in a distributed environment introduces concurrency hazards. During an insertion operation, the filter performs a sequence of displacements that modify multiple bucket locations. A concurrent query examining these locations during this transient state may fail to find an element that has been evicted from its original bucket but not yet placed in its alternate location. This race condition, which we term a *false miss*, results in a definitive negative answer for an element present in the set, thereby violating the data structure's membership guarantee.

We analyze three distinct concurrency control strategies, each designed to resolve the false-miss problem for a specific system architecture and workload pattern.

The first strategy, optimistic concurrency, is designed for read-dominant workloads, such as those found in caching systems. Fan et al. developed this model for the MemC3 system [13]. The model serializes write

operations while allowing multiple readers to proceed without locks, using version counters for synchronization. A key component of this model is the execution order of relocations along a displacement chain. After identifying a valid path, the system moves the last element in the chain into the empty slot first, which frees a new slot. The second-to-last element is then moved into this newly vacated slot. This reverse-order execution ensures that every element being relocated remains accessible in at least one of its two candidate buckets at all times.

Definition 2.21 (Optimistic Cuckoo Hashing [13])**.** A writer maintains a version counter for each bucket. Before initiating relocations, it identifies a valid displacement path (e.g., $a \to b \to c \to$ empty) without acquiring locks. The relocations are then executed in reverse order, with each move incrementing the version counter of the involved bucket. A reader records the versions of its two candidate buckets before inspection and rechecks them afterward. If any version has changed, the reader discards its result and retries the operation.

This strategy is highly effective for workloads where the read-to-write ratio is high, as readers do not incur lock acquisition overhead.

For general-purpose multi-core systems with more balanced read-write workloads, we analyze a fine-grained locking model that enables greater writer parallelism. The libcuckoo library[1] implements this model using lock striping, where an array of independent locks protects disjoint sets of buckets [14, 15]. A thread acquires locks only for the buckets involved in its operation, which permits concurrent write operations on non-overlapping regions of the filter.

For disaggregated memory architectures that utilize Remote Direct Memory Access (RDMA), in-memory locking mechanisms are inapplicable. Clients access remote memory directly over the network. Grant et al. developed RCuckoo for this environment, which employs a lease-based locking protocol [16]. In this model, clients acquire time-bounded leases on remote buckets via atomic RDMA operations. The automatic expiration of these leases provides fault tolerance to client failures without requiring explicit recovery protocols.

Definition 2.22 (Lease-Based Locking for RDMA [16])**.** Each bucket in remote memory maintains a lease counter and an expiration timestamp. A client requests a lease by issuing an RDMA compare-and-swap operation that atomically increments the counter, succeeding only if the current time exceeds the previous lease's expiration. Upon acquiring the lease, the client records a new expiration time, performs its operation, and subsequently releases the lease. If a client fails, the lease expires after its designated duration, allowing other clients to acquire it.

We note that across all three concurrency models, the no-false-negatives property of the Cuckoo Filter remains invariant. A query that returns false provides a correct answer: the element is not, and has never been (unless subsequently deleted), a member of the set. The specific concurrency strategy is an implementation detail dictated by the hardware topology and workload, but the membership guarantee is preserved.

2.2.7 Replication and Convergence in Eventually Consistent Systems

When we replicate Cuckoo Filters across multiple nodes for fault tolerance, the replicas may temporarily diverge due to network latency in propagating updates. We analyze the consequences of this divergence, particularly for deletion operations, and compare the behavior of Cuckoo Filters to that of standard and Counting Bloom Filters.

A standard Bloom filter provides no mechanism for deletion; elements, once inserted, remain in the filter indefinitely. A Counting Bloom Filter supports deletion by decrementing counters but at the cost of an approximately four-fold increase in space overhead [8]. A Cuckoo Filter, by contrast, implements deletion as a direct removal of the element's fingerprint.

In a replicated environment, this distinction becomes architecturally significant. Consider a Cuckoo Filter replicated on nodes n_1 and n_2. An insertion of element x on n_1 propagates asynchronously to n_2. During the propagation window, a query for x on n_2 will return false. This behavior, a form of temporal false negative, is inherent to any asynchronously replicated system.

The key difference emerges in deletion scenarios, a problem central to systems that use tombstones to mark deleted data under eventual consistency [17]. When element x is deleted on n_1, its fingerprint is immediately removed from the local filter. During the propagation window, a query for x to n_1 correctly returns false, while a query to n_2 returns a stale positive.

[1] `https://github.com/efficient/libcuckoo`

Once the deletion operation propagates to n_2, its fingerprint is also removed, and both nodes converge to a consistent state where x is correctly identified as absent.

A Bloom filter behaves differently. Since it cannot remove the element, it will continue to return positive for the deleted key indefinitely. This forces the system to perform expensive validation lookups against the backing data store merely to discover a tombstone, negating much of the filter's performance benefit in workloads with frequent deletions [17].

Proposition 2.23 (Cuckoo Filter Replication Convergence)**.** *In an eventually consistent system with replicated Cuckoo Filters, let an element x be inserted at time t_i and deleted at time $t_d > t_i$. For any replica r, there exist propagation delays such that the insertion is applied at time $\tau_i \geq t_i$ and the deletion is applied at time $\tau_d \geq t_d$. For all times $t > \tau_d$, a query for x on replica r will deterministically return false. In contrast, a standard Bloom filter provides no upper bound on the time until a query for a deleted element ceases to return a positive result.*

This property enables Cuckoo Filters to converge more rapidly to the true state of the system, especially in environments with frequent deletions. For distributed systems implementing eventual consistency, this allows the filter's state to accurately track operations such as the purging of tombstones during compaction or garbage collection. The ability to remove fingerprints, rather than merely marking them for future cleanup, ensures that the filter remains a reliable oracle for the live, non-deleted state of the dataset.

2.2.8 Impact on Distributed System Architectures

We analyze the concrete architectural impact of Cuckoo Filters in several domains where distributed systems require both efficient membership testing and dynamic element deletion.

Our first application domain is Log-Structured Merge-Tree (LSM-tree) based key-value stores. In these systems, data is organized into immutable, sorted files on disk (SSTables), each typically indexed by an in-memory filter. Deletions are often handled by writing a tombstone marker rather than by immediate data removal [17]. A standard Bloom filter, being unable to remove entries, continues to return a positive match for a deleted key. This behavior forces the database to perform a disk I/O operation only to discover the tombstone, incurring unnecessary latency. A Cuckoo Filter resolves this inefficiency. During data compaction, the fingerprint corresponding to a purged tombstone can be removed from the filter. Subsequent queries for the deleted key will correctly return false, eliminating the wasted I/O. The Chucky architecture further exploits this by unifying the multiple per-level filters of an LSM-tree into a single Cuckoo Filter, which reduces the point query complexity from O(number of levels) to $O(1)$ [18].

In named-data networking (NDN), we examine the Pending Interest Table (PIT), which tracks outstanding content requests. The DiCuPIT system utilizes distributed Cuckoo Filters to replace traditional Bloom filter-based implementations. This architectural change yields a 36% reduction in lookup time and a 31% reduction in memory consumption compared to Bloom filter baselines [19]. The primary advantage is the ability to correctly and efficiently delete PIT entries as their corresponding data responses arrive, a fundamental operation that standard Bloom filters cannot support.

Our third domain is state synchronization in blockchain networks. Nodes in these systems maintain a mempool of pending transactions. To propagate a new block efficiently, the Gauze protocol employs Cuckoo Filters to summarize the contents of a node's mempool [20]. A node propagating a block sends this compact filter to its peers. The receiving nodes use the filter to identify which transactions in the new block are absent from their local mempool and request only this delta. This method reduces block propagation bandwidth by 40 to 50%. The efficiency gain is a direct result of the filter's compact representation; a filter for a million-element set requires only a few megabytes of space.

These applications demonstrate a shared architectural requirement: efficient support for both insertion and deletion in resource-constrained environments, where the primary bottlenecks are network bandwidth, I/O latency, or processing throughput. The ability to delete elements distinguishes the Cuckoo Filter from the Bloom filter not as an incremental improvement, but as an enabling architectural property. It facilitates correct state reconciliation, efficient rebalancing under dynamic scaling, and accurate membership semantics in systems where the lifecycle of data includes explicit deletion.

References

[1] Aravind Sekar. The future of large-scale distributed systems: Trends, challenges, and opportunities. 11:3374–3390, Apr. 2025.

[2] Helen Mayer and James Richards. Comparative analysis of distributed caching algorithms: Performance metrics and implementation considerations. *arXiv preprint arXiv:2504.02220*, 2025.

[3] Baozhou Luo, Wenjun Zhu, Peng Li, and Zhijie Han. Distributed dynamic cuckoo filter system based on redis cluster. In *2018 IEEE 4th International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing,(HPSC) and IEEE International Conference on Intelligent Data and Security (IDS)*, pages 244–247. IEEE, 2018.

[4] Lum Ramabaja and Arber Avdullahu. The distributed bloom filter. *arXiv preprint arXiv:1910.07782*, 2019.

[5] Sasu Tarkoma, Christian Esteve Rothenberg, and Eemil Lagerspetz. Theory and practice of bloom filters for distributed systems. *IEEE Communications Surveys & Tutorials*, 14(1):131–155, 2011.

[6] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm. *Discrete mathematics & theoretical computer science*, (Proceedings), 2007.

[7] Stefan Heule, Marc Nunkesser, and Alexander Hall. Hyperloglog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm. In *Proceedings of the 16th International Conference on Extending Database Technology*, pages 683–692, 2013.

[8] Bin Fan, David G Andersen, Michael Kaminsky, and Michael D Mitzenmacher. Cuckoo filter: Practically better than bloom. In *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies*, pages 75–88, 2014.

[9] Hanhua Chen, Ligang Liao, Hai Jin, and Jie Wu. The dynamic cuckoo filter. In *Proceedings of the 2017 IEEE International Conference on Network Protocols (ICNP)*, pages 1–10. IEEE, 2017.

[10] Apache Software Foundation. Apache cassandra documentation: Consistent hashing, 2014.

[11] Redis Labs. Redis cluster specification, 2020.

[12] Lailong Luo. Capacity elastic Cuckoo Filter design. 2021.

[13] Bin Fan, David G Andersen, Michael Kaminsky, and Michael D Mitzenmacher. Memc3: Compact and concurrent memcache with dumber caching and smarter hashing. In *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, pages 371–384, 2013.

[14] Xiaozhou Li, David G Andersen, Michael Kaminsky, and Michael D Mitzenmacher. Algorithmic improvements for fast concurrent cuckoo hashing. In *Proceedings of the 9th European Conference on Computer Systems*, pages 1–14, 2014.

[15] Wenhai Li. Lock free bucketized Cuckoo Hashing. 2023.

[16] Evan Grant et al. Cuckoo for clients: Disaggregated cuckoo hashing. In *2025 USENIX Annual Technical Conference (ATC 25)*, 2025.

[17] Sharafat Ibn Mollah Mosharraf and Muhammad Abdullah Adnan. Improving lookup and query execution performance in distributed big data systems using cuckoo filter. *Journal of Big Data*, 9(1):12, 2022.

[18] Niv Dayan. Chucky: A succinct cuckoo filter for lsm-trees. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, 2020.

[19] Arman Mahmoudi and Mahmood Ahmadi. Dicupit: Distributed cuckoo filter-based pending interest table. *arXiv preprint arXiv:2308.02801*, 2023.

[20] Taeyoon Kim, Young-Chul Kim, Yun-Hyeong Kim, and Hyoungshick Kim. Gauze: enabling communication-friendly block synchronization with cuckoo filter. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1755–1769, 2022.

[21] Steven D Gribble, Eric A Brewer, and Joseph M Hellerstein. Scalable, distributed data structures

for internet service construction. In *Fourth Symposium on Operating Systems Design and Implementation (OSDI 2000)*, 2000.

[22] Pedro Reviriego and Salvatore Pontarelli. Perfect cuckoo filters. In *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies*, pages 75–88, 2021.

[23] Efficient Computing Laboratory. libcuckoo: A high-performance, concurrent hash table, 2016.

[24] Ross Devine. Design and implementation of ddh: A distributed dynamic hashing algorithm. In *Foundations of Data Organization and Algorithms*, pages 101–114. Springer, 1997.

[25] Anonymous. A survey of cuckoo filters and hashing in distributed environments, 2024. Provided survey material.

[26] Poojan. In memory data filtering Cuckoo Hashing.

[27] David Eppstein. Cuckoo Filter simplification analysis. 2016.

[28] Lang. Performance optimal filtering Bloom overtakes Cuckoo. 2019.

[29] Wang. Vacuum Filters space efficient faster replacement. 2019.

[30] Michael Mitzenmacher. Adaptive Cuckoo Filters. 2020.

[31] Yihan Sun. Snapshot isolation hybrid workloads multi versioned indexes. 2019.

[32] Niv Dayan. Chucky succinct Cuckoo Filter LSMTree. 2021.

[33] Aman Khalid. Optimized Cuckoo Filters SDN NFV. 2020.

[34] Jan Grashöfer. Cuckoo Filters network security monitoring. 2018.